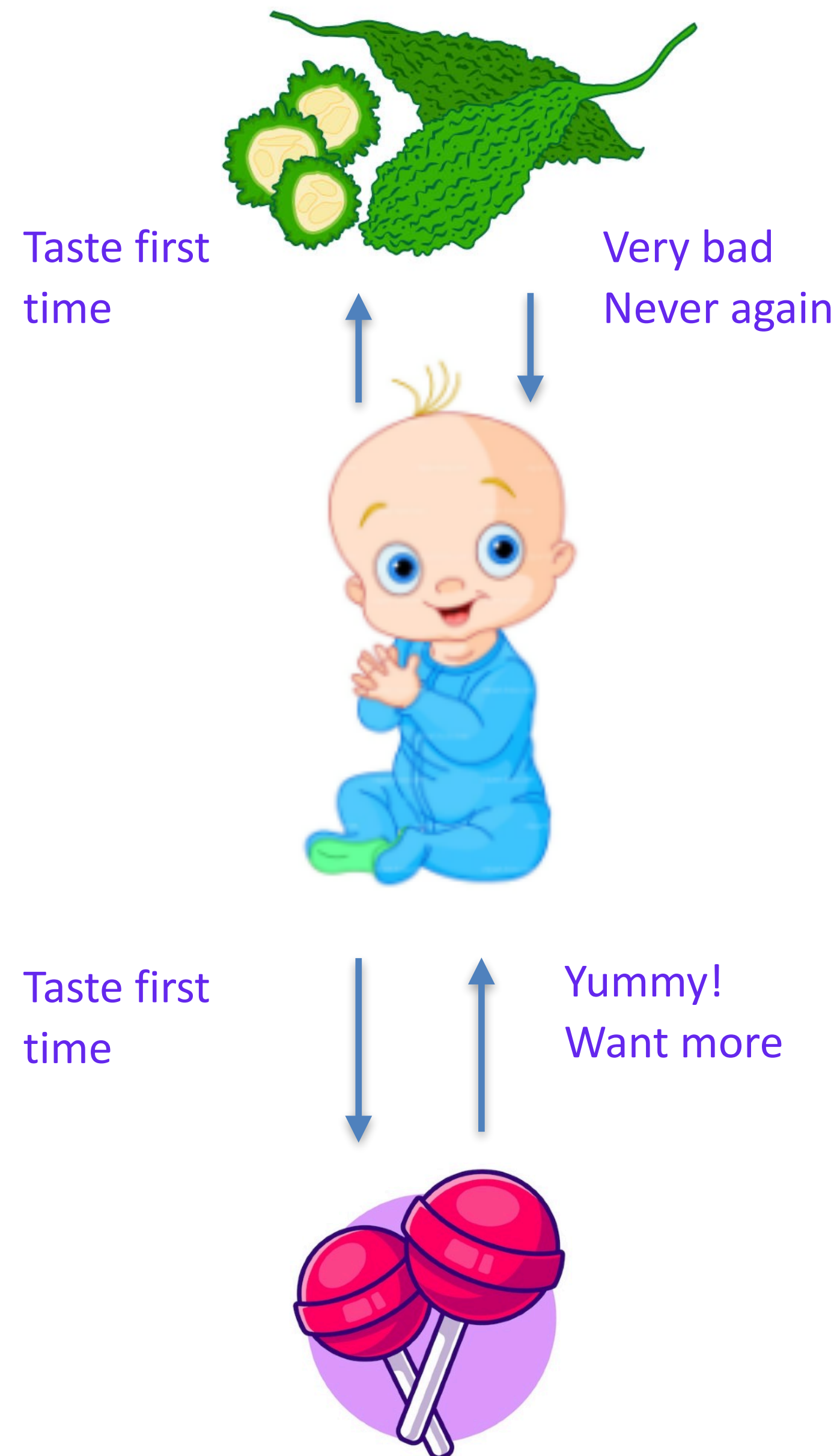


Reinforcement Learning

Debapriyo Majumdar
Indian Statistical Institute
debapriyo@isical.ac.in

A Human Analogy



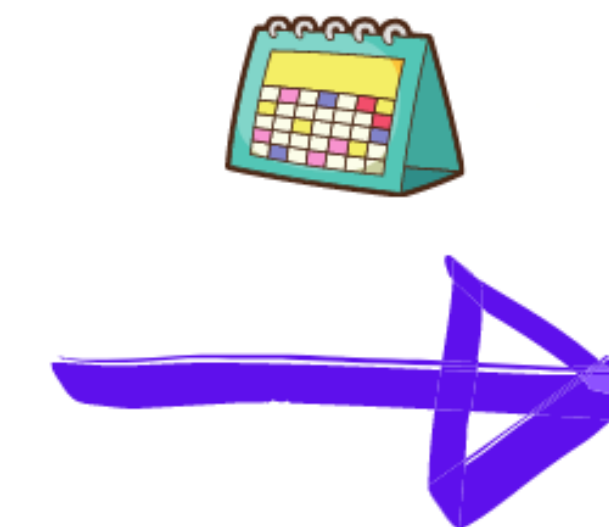
Desired
Behaviour



Reward



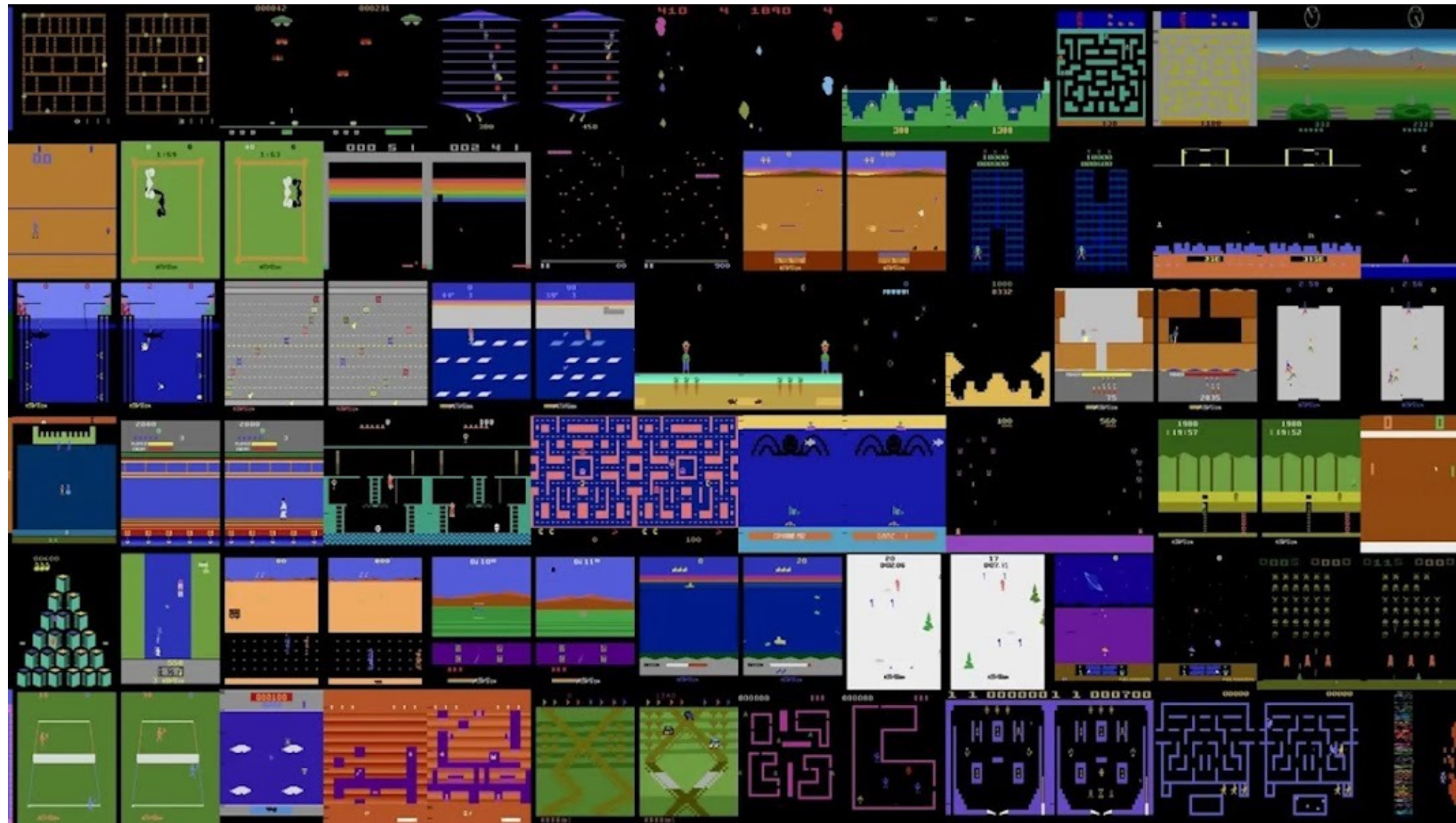
More Desired
Behaviour



With time
meaningful change
in behavior happens

Why is Reinforcement Learning Popular Now?

3



Agent 57: beating human in Atari games

Also being applied to
finance, dialogues (NLP), recommendation, ...



Alpha Go: Beats world champion in Go



Computational Robotics

The Basic Paradigm

Supervised learning

Learn a function given a fixed dataset



cat!



dog!

f



x

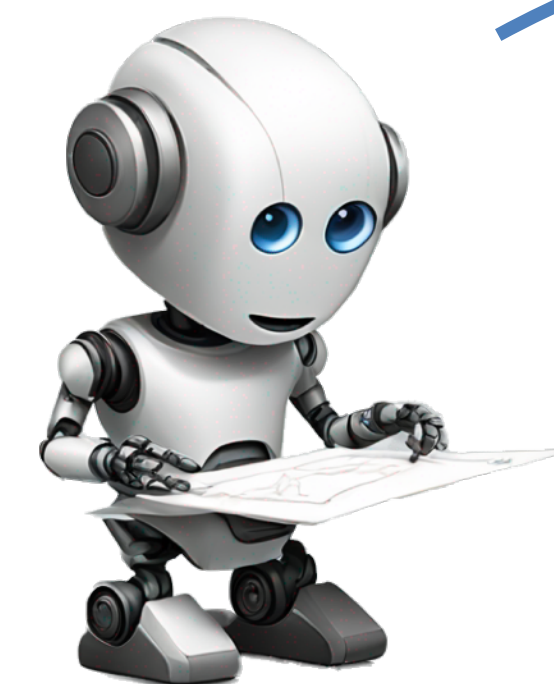
$\hat{y}$

input

output

Reinforcement learning

Learn a decision making policy from experience

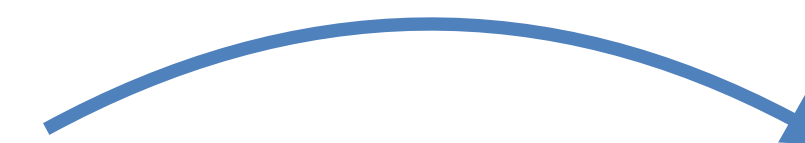


agent



environment

take action



get feedback



Learns to take better action

What is Reinforcement Learning?

Supervised Learning

Have labeled data (training data)
“Situation” and “label” (correct answer for the task)
Have to learn how to answer for unknown inputs.

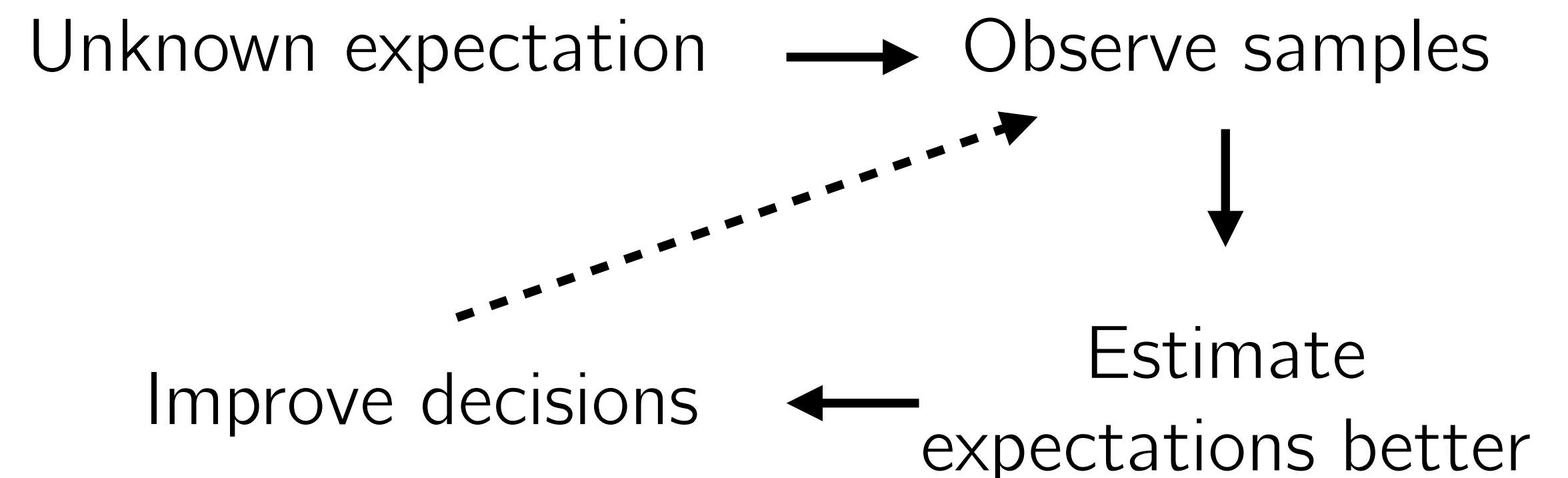
Unsupervised Learning

Unlabelled data
Have to find structures hidden in the collection

■ Reinforcement Learning

- Given situations and actions
- Task: learn *what to do*
- Learner must try out actions and discover the consequences
- Get better over time / many actions

A recurrent theme we will see



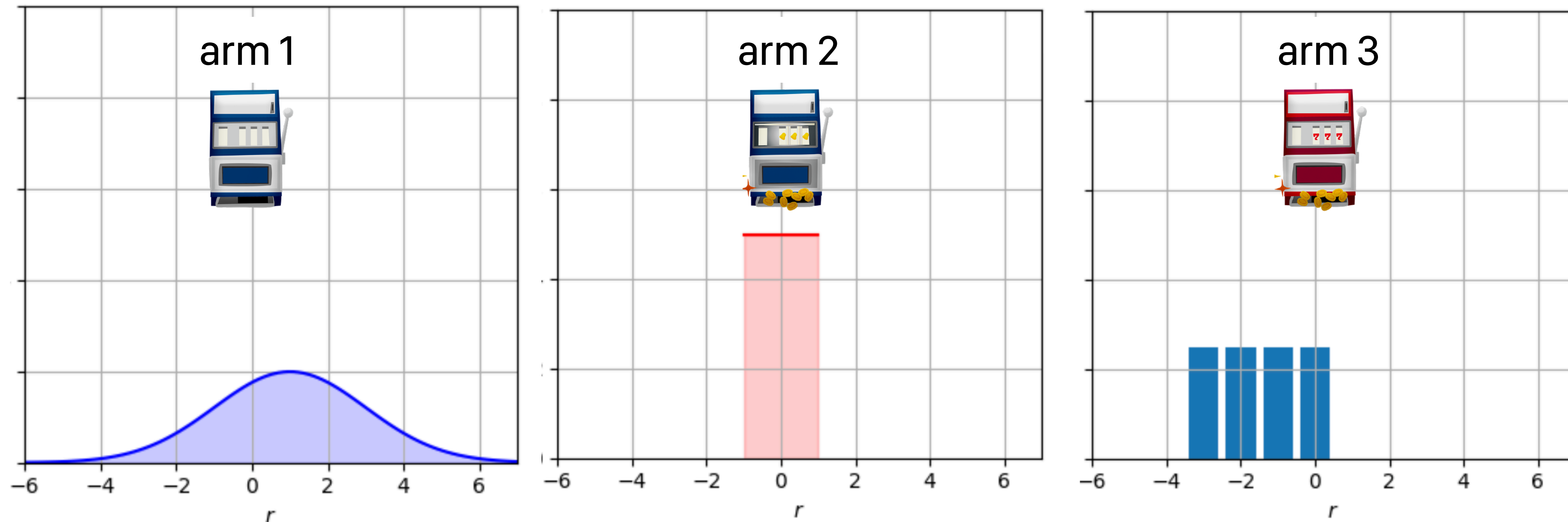
Tentative Course Structure

1. Tabular solution methods (9-10 lectures)
2. Function approximation methods (7-8 lectures)
3. Deep Reinforcement Learning and modern trends (9-10 lectures)

References

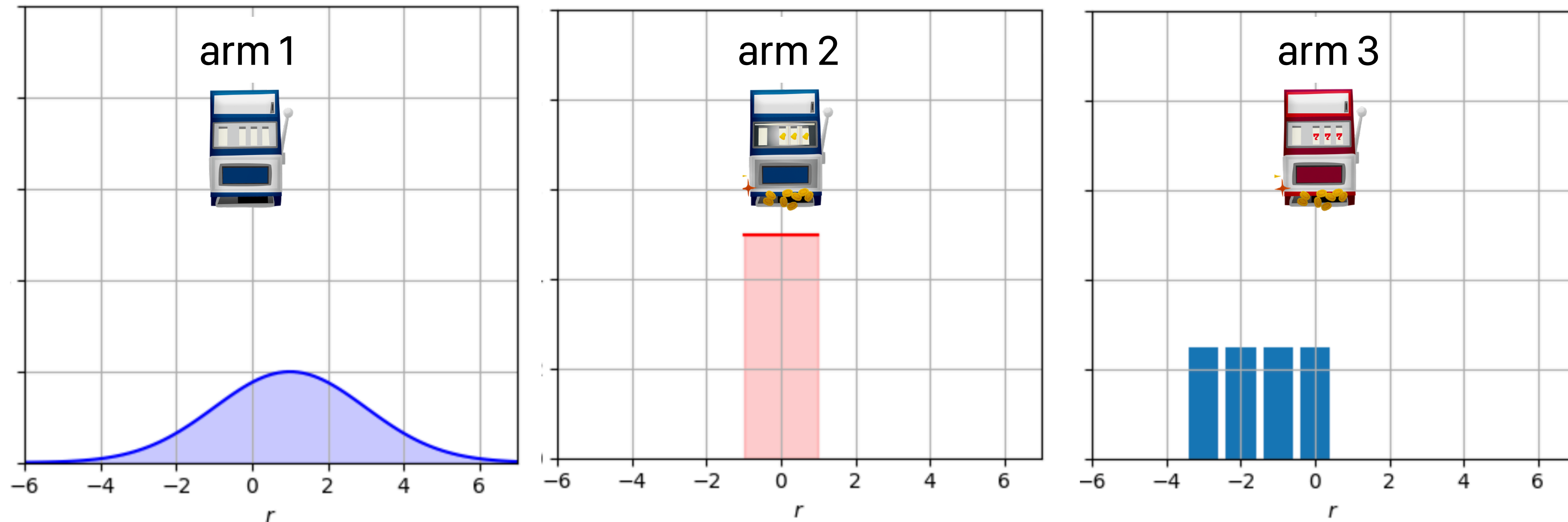
- Sutton, Richard S., and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018. <http://incompleteideas.net/book/the-book-2nd.html>
- François-Lavet, Vincent, Peter Henderson, Riashat Islam, Marc G. Bellemare, and Joelle Pineau. "An introduction to deep reinforcement learning." *arXiv preprint arXiv:1811.12560*(2018). <https://arxiv.org/pdf/1811.12560.pdf%3C/p%3E>
- Agarwal, Alekh, Nan Jiang, Sham M. Kakade, and Wen Sun. "Reinforcement learning: Theory and algorithms." *CS Dept., UW Seattle, Seattle, WA, USA, Tech. Rep* (2019). <https://rltheorybook.github.io>
- Open AI Spinning Up: <https://spinningup.openai.com/en/latest/>

The Multi-Arm bandit problem



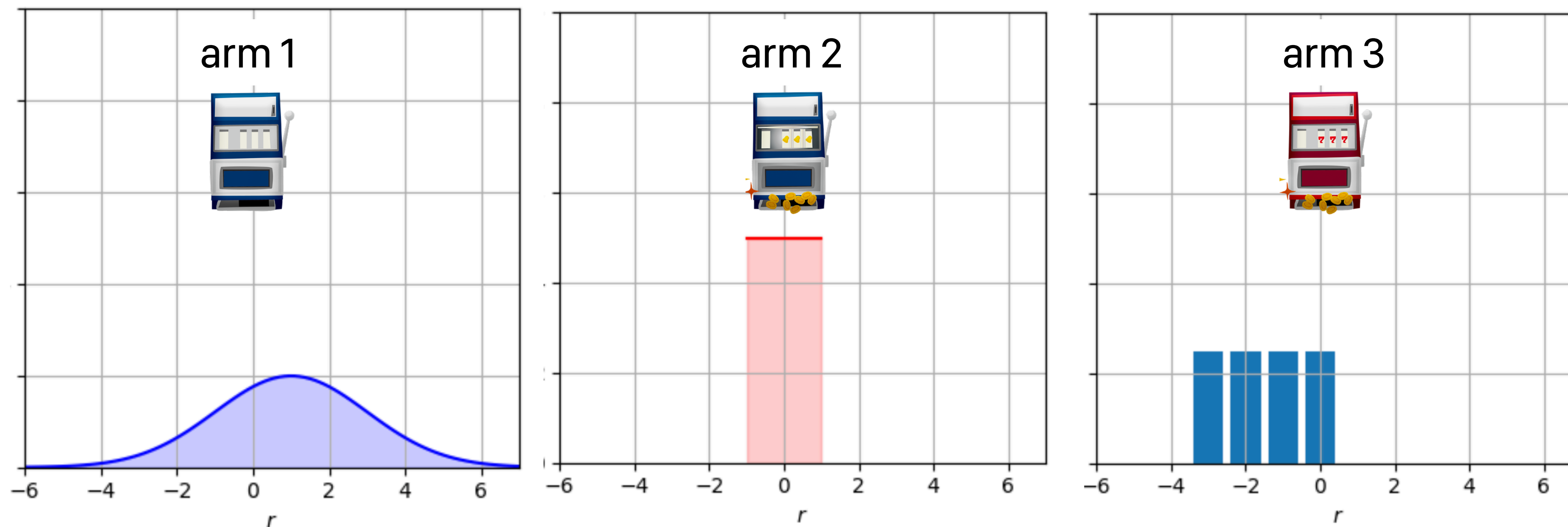
- There are k slot machines (arms), each with a **stationary probability distribution** of rewards
 - The distributions are not known to us to start with
- At each step t , we choose to draw from one of the k slot machines
 - Get a **reward** $R(t)$
- Goal: maximize total reward over a time period (say N actions)

The Multi-Arm bandit problem



- **Value** of an action: Expected (mean) reward for an action a is $q_*(a) = \mathbb{E}(R_t | A_t = a)$
- However, we don't know $q_*(a)$, we need to estimate by trials and obtained rewards
- Define $Q_t(a)$ = estimated value of action a at step t
- Goal: try to make $Q_t(a)$ as close to $q_*(a)$ as possible, as quickly as possible

The Multi-Arm bandit problem



- Sample average method: estimate $Q_t(a)$ the value of an action a by averaging the rewards actually received from action a at time t

$$Q_t(a) = \frac{\text{total rewards when action } a \text{ is taken prior to } t}{\text{number of times } a \text{ is taken prior to } t}$$

- The value is some default (say 0) if action a has not been taken yet (denominator is 0)

Why does that work?

- A consequence of the ***law of large numbers***

- Suppose

$$X_1, X_2, \dots, X_n$$

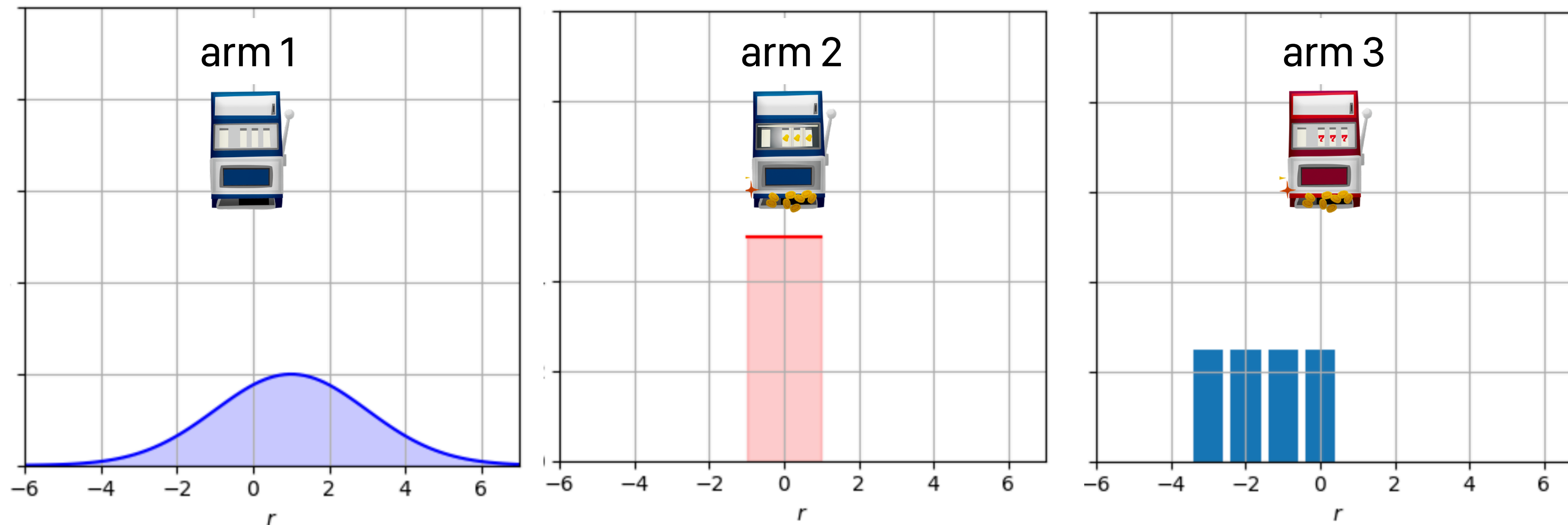
are independent observations (samples) of a random variable X following some distribution

- Then

$$\frac{1}{n} \sum_{i=1}^n X_i \longrightarrow \mathbb{E}[X] \quad \text{as } n \rightarrow \infty$$

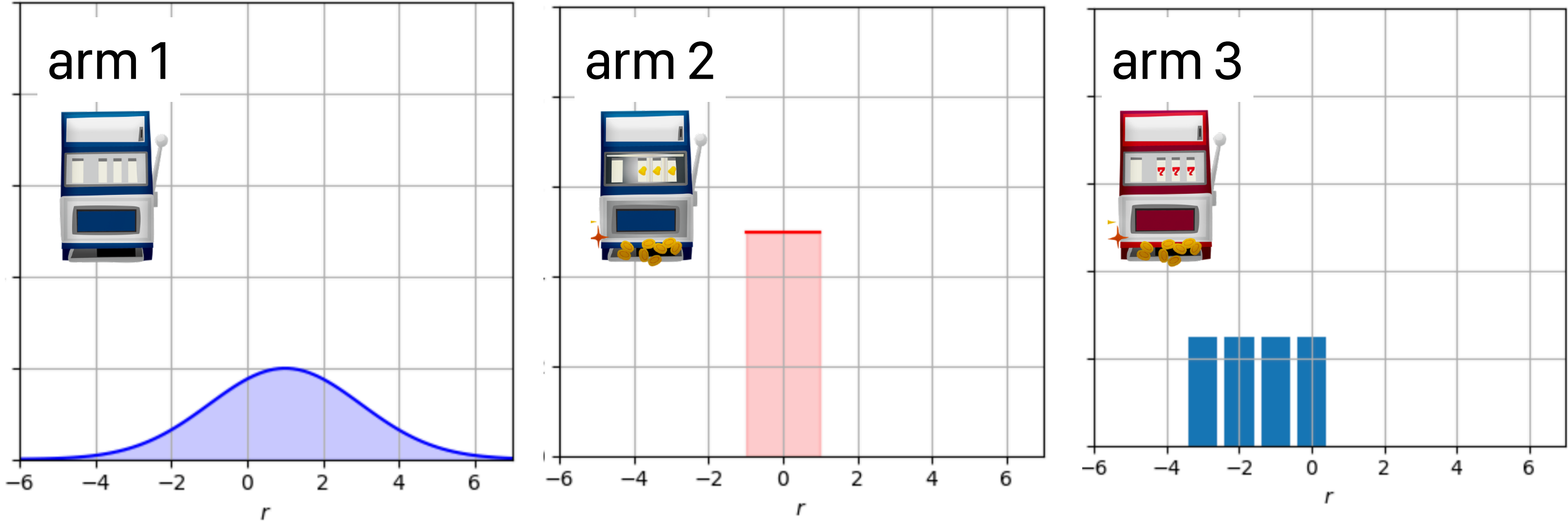
- As we collect more samples, the ***average*** stabilizes around the ***true expectation***

Simplest: The Greedy Algorithm



- Initialize $Q(a) = 0$ for all actions (arms)
- Repeat
 - Choose the greedy (one with the highest current estimate) action
 - Get reward, update the estimate (average reward of that action so far)

Greedy Algorithm: Example



Greedy action selection

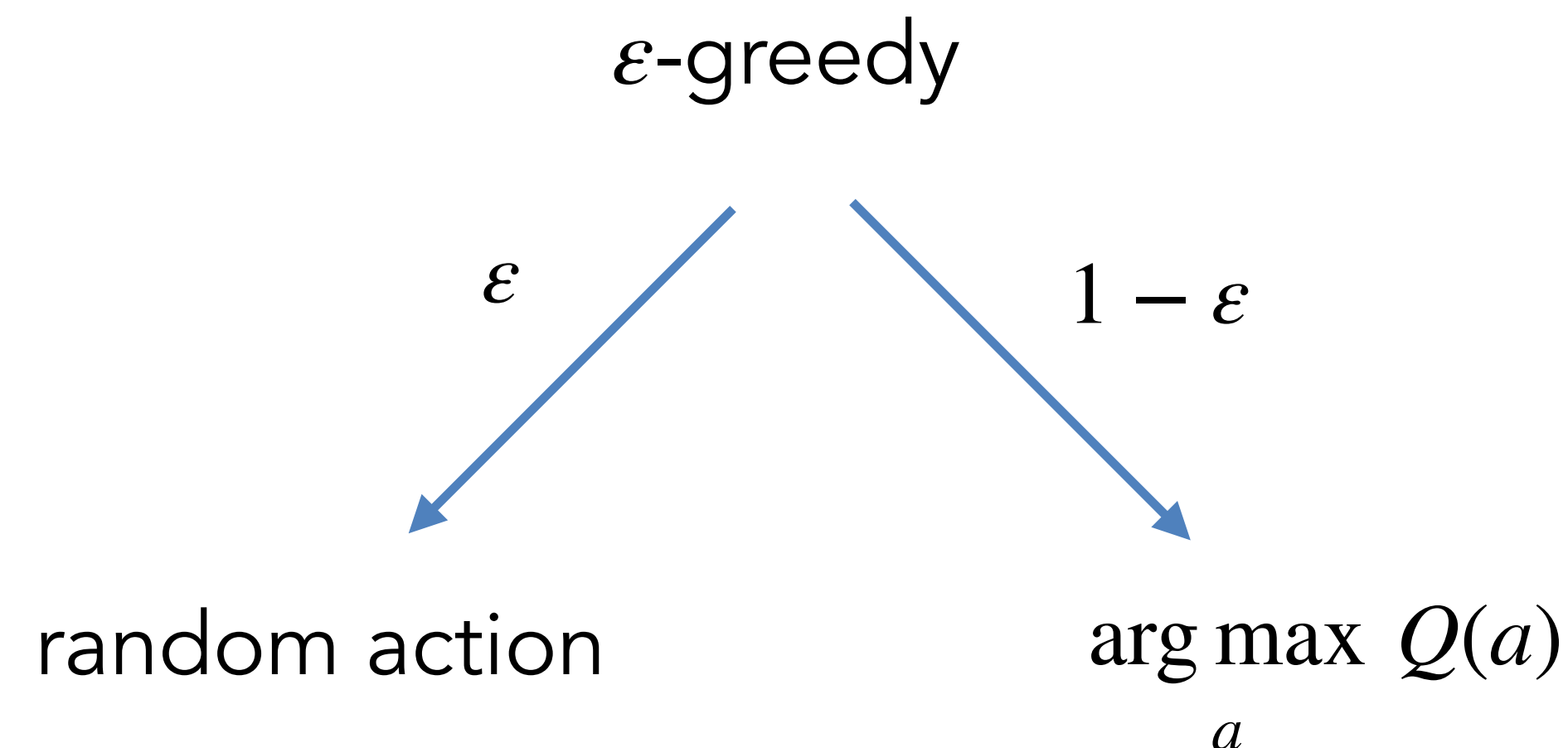
$$A_t = \arg \max_a Q_t(a)$$

$$Q(a) = Q(a) + \frac{1}{N(a)}(R - Q(a))$$

$$N(A) = \text{number of times action } a \text{ taken}$$

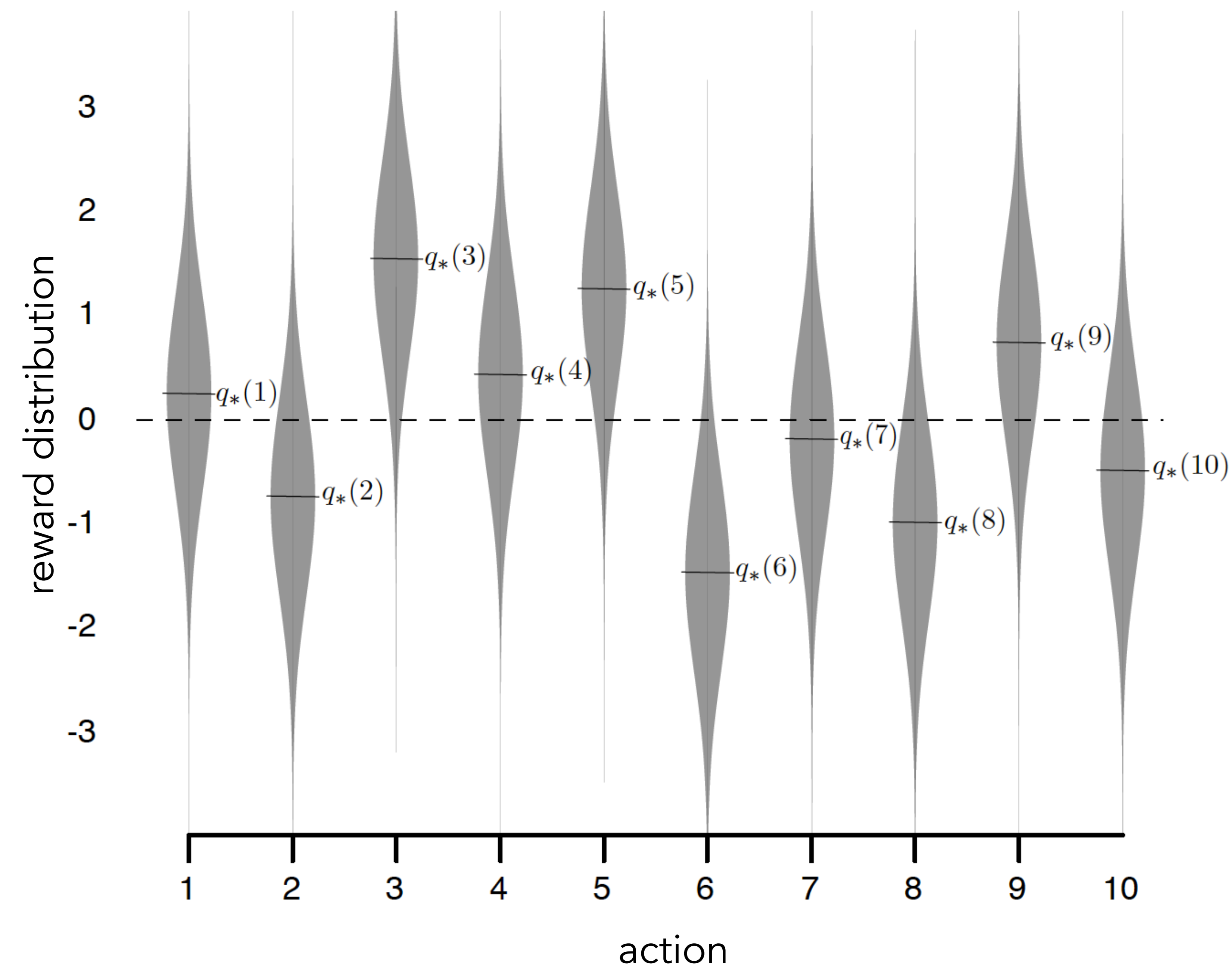
Step	0	1	2	3	4	5				Eventually
Action		1	2	2	3	2				
Reward		-0.5	0.1	-0.9	-1	-1				
Q(1)	0	-0.5	-0.5	-0.5	-0.5	-0.5	after many steps			1
Q(2)	0	0	0.1	-0.4	-0.4	-0.6				0
Q(3)	0	0	0	0	-1	-1				-1.5

The ϵ -Greedy Approach



- Exploitation (greedy approach): at each step t , pick the action a for which the estimate $Q_t(a)$ is highest
 - Some actions may get **permanently rejected unduly**
- Exploration: pick some non-greedy action
 - May help improving estimates better
 - May maximize reward in the long run
- Balance exploitation and exploration
 - Advantage: eventually all actions will be taken "many" times

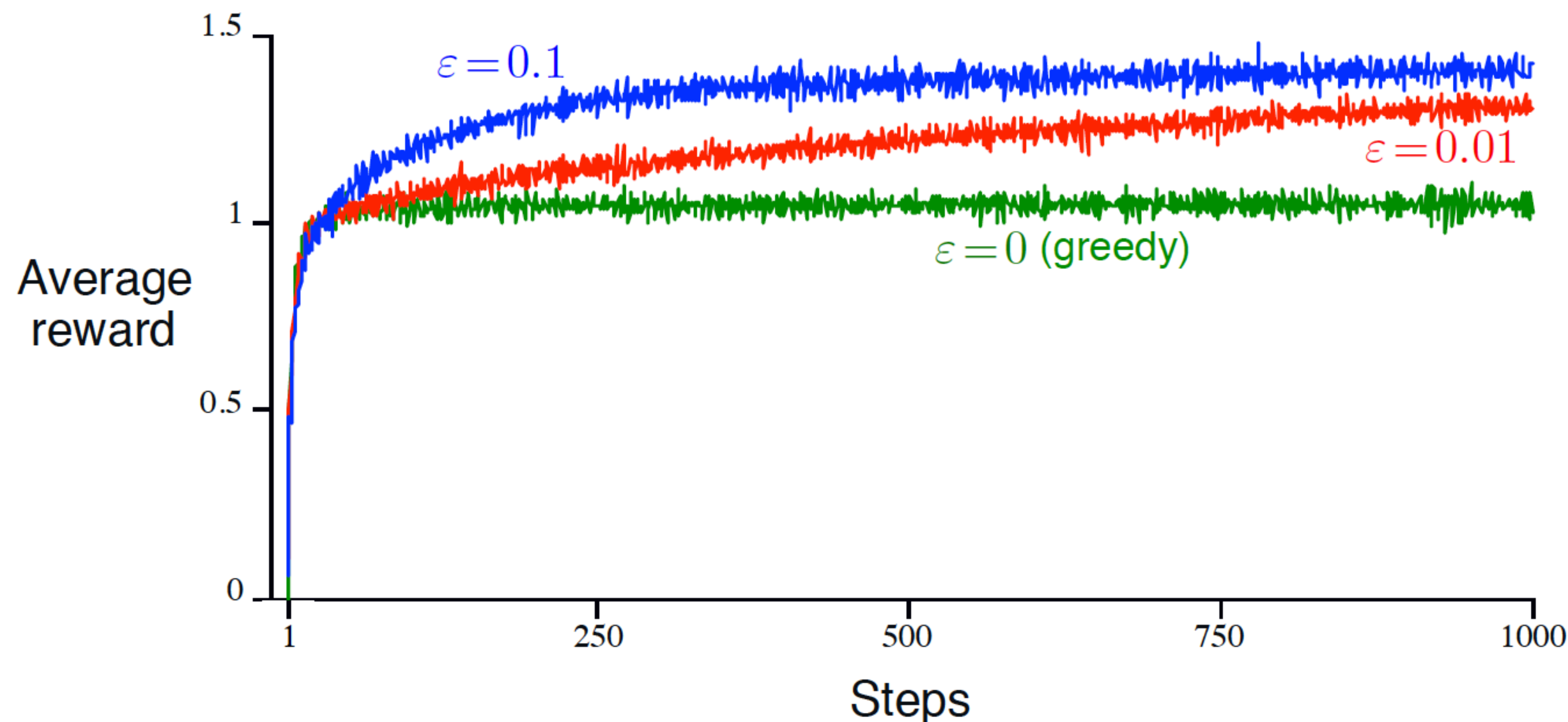
Experiment: k -armed testbed ($k = 10$)



- Generate k random numbers from $\mathcal{N}(0,1)$ as $q_*(a)$ for $a = 1, 2, \dots, k$
- Consider a k -armed bandit where each action a has reward distribution following mean $q_*(a)$ and variance 1, i.e., following $\mathcal{N}(q_*(a), 1)$
- Simulate different algorithms on this testbed
- One particular simulation is not reliable, so consider average performance over many (say 2000) bandits generated this way

Greedy vs ϵ -greedy

- Greedy performs better initially
- However, ϵ -greedy methods performs more explorations and eventually outperforms greedy
- Eventually, $\epsilon = 0.01$ would outperform the other two (why?)



Picture courtesy: Sutton and Barto, 2018

General form of the update rule

New estimate = Old estimate + Step size (Target – Old estimate)

$$Q(a) = Q(a) + \alpha_t(a)[R_t(a) - Q(a)]$$

For the algorithm we have seen, the step size

$$\alpha_t(a) = \frac{1}{N(a)}.$$

Non-stationary bandit problem

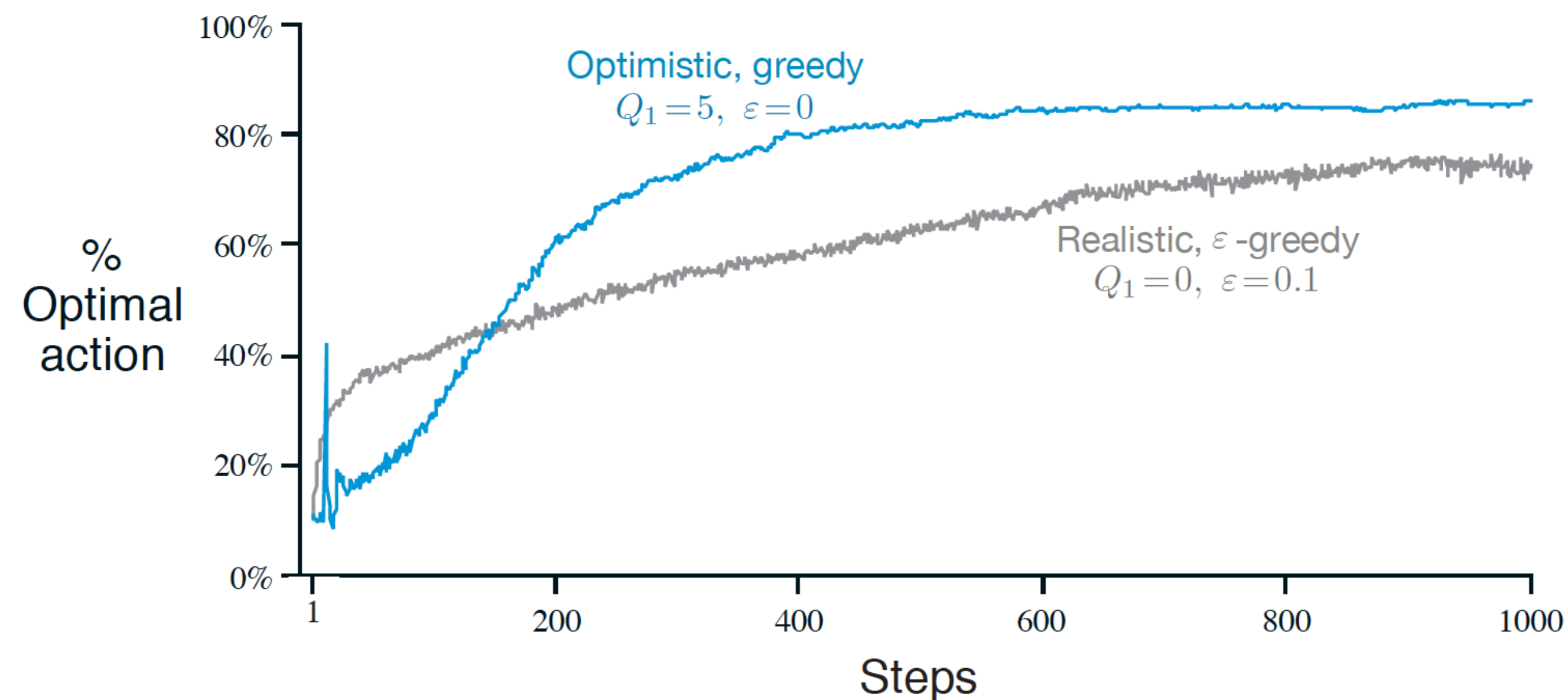
- Non-stationary probability: the reward probability distribution may change over time
 - More towards real world
- Need to have higher weightage for more recent rewards
- One possible approach: **exponential recency-weighted average**
 - Can be achieved by a constant step size (instead of a decaying step size) $\alpha_t(a) = \alpha$
- Then:
$$Q_{n+1} = Q_n + \alpha[R_n - Q_n] = \alpha R_n + (1 - \alpha)Q_n$$
- Using this repeatedly by backward induction, we obtain
$$Q_{n+1} = (1 - \alpha)^n Q_1 + \alpha(1 - \alpha)^{n-1} R_1 + \alpha(1 - \alpha)^{n-2} R_2 + \dots + \alpha(1 - \alpha) R_{n-1} + \alpha R_n$$
- Weightage for older Q_i decreases exponentially

Convergence of estimates to the actual value

- Goal: We want $Q_n(a) \rightarrow q_*(a)$ for large n
- Result: using the update rule $Q(a) = Q(a) + \alpha_t(a)[R_t(a) - Q(a)]$, the estimate converges with probability 1 if,
 - The steps are large enough to overcome initial effect of random fluctuations: $\sum_{t=1}^{\infty} \alpha_t(a) = \infty$, and
 - The steps are small enough to ensure convergence: $\sum_{t=1}^{\infty} \alpha_t^2(a) < \infty$
- For step size $\alpha_t(a) = \frac{1}{t}$, the conditions are satisfied, the estimates converge
- For step size $\alpha_t(a) = \alpha$ (constant), the second condition is not satisfied
 - The estimates continue to vary with recent fluctuations
 - But, that is desirable in a non-stationary environment

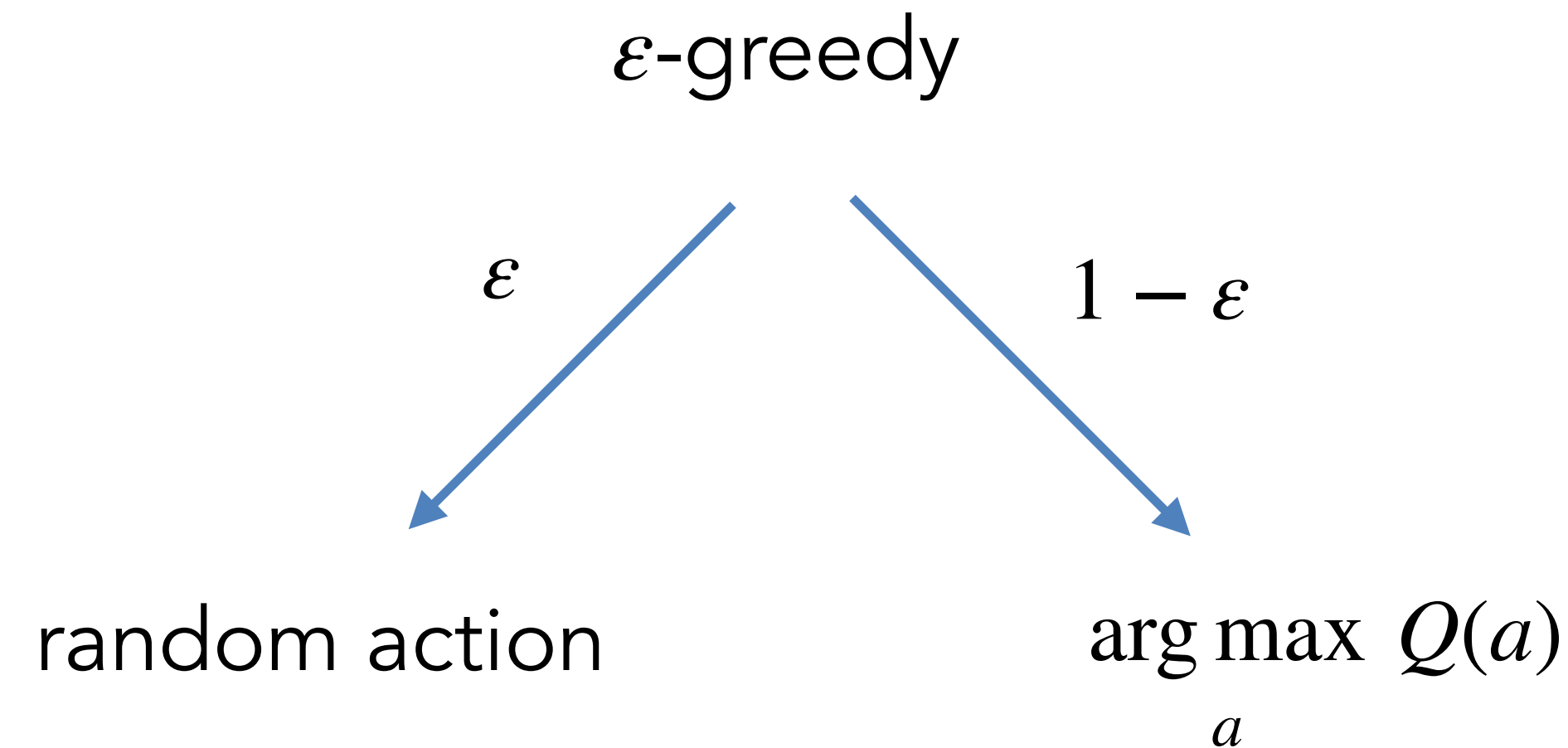
Optimistic initial value

- Another approach for *encouraging* exploration: have high initial values for rewards
- Zero initialization: all estimates start with zero
- Optimistic initialization: initialize all estimates to some number much higher than $q_*(a)$ for all a
 - Time $t = 1$: some action will be chosen, but the value will be $<$ the estimate (high initialization)
 - Disappointed: will choose some other action next time (but will be disappointed again)
 - During the initial stage, will ensure all actions are explored (but that's temporary)



Average performance of optimistic greedy vs realistic ϵ -greedy

Upper confidence bound (UCB) action selection



- In ϵ -greedy, the random action selection gives same preference to all actions
 - However, the goal of the non-greedy action is to *explore*
 - Reduce the uncertainty about the estimate of the action values
 - At any step t , the uncertainty about all actions may not be the same
 - Also, not all actions have the same potential to be maximal

Upper confidence bound (UCB) action selection

- Choose action based on the potential of that being maximal

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

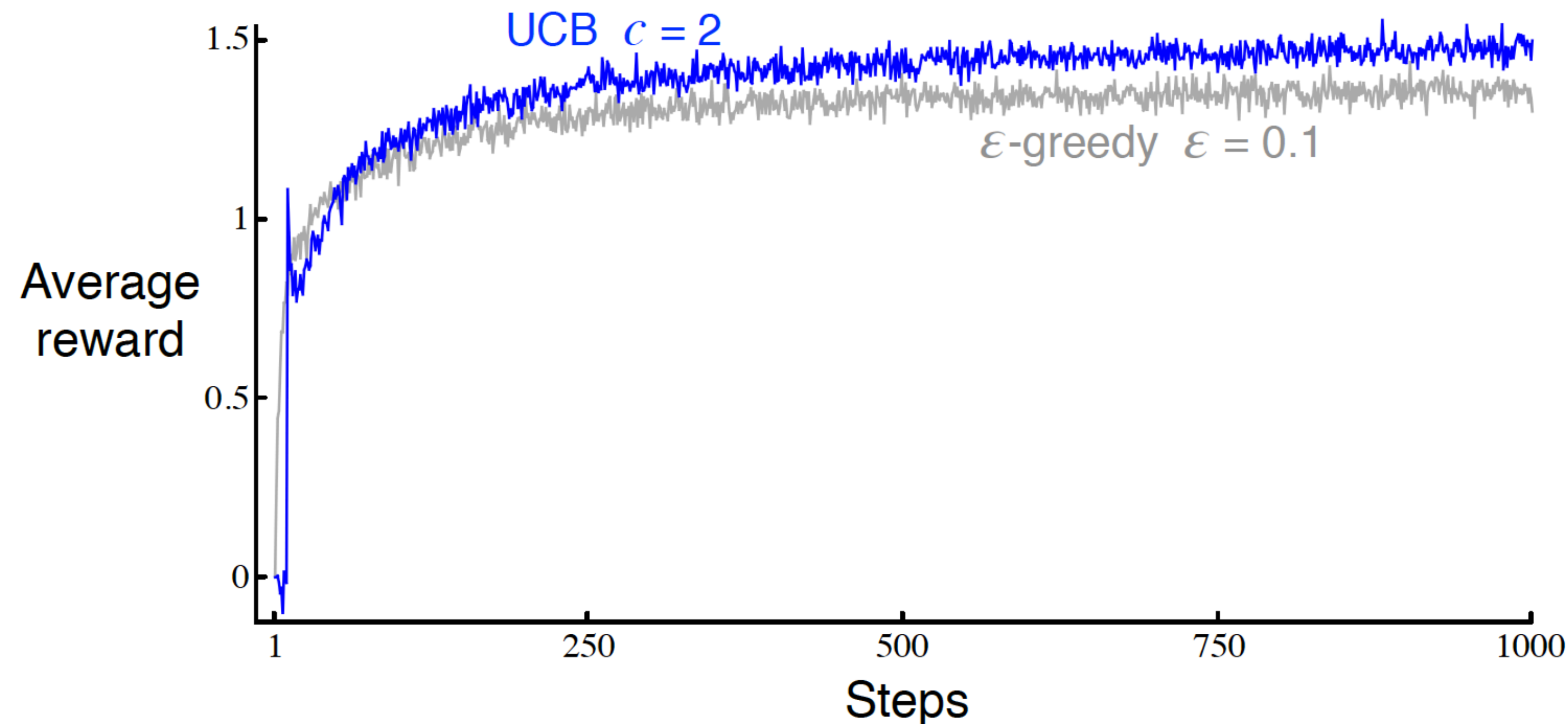
Already estimated value.
More the estimate more the potential.

A measure of uncertainty of $Q_t(a)$.
If $N_t(a)$ is large, action a has been taken many times already.

c : Exploration factor.
 $c = 0$ is equivalent to greedy.

Upper confidence bound (UCB) action selection

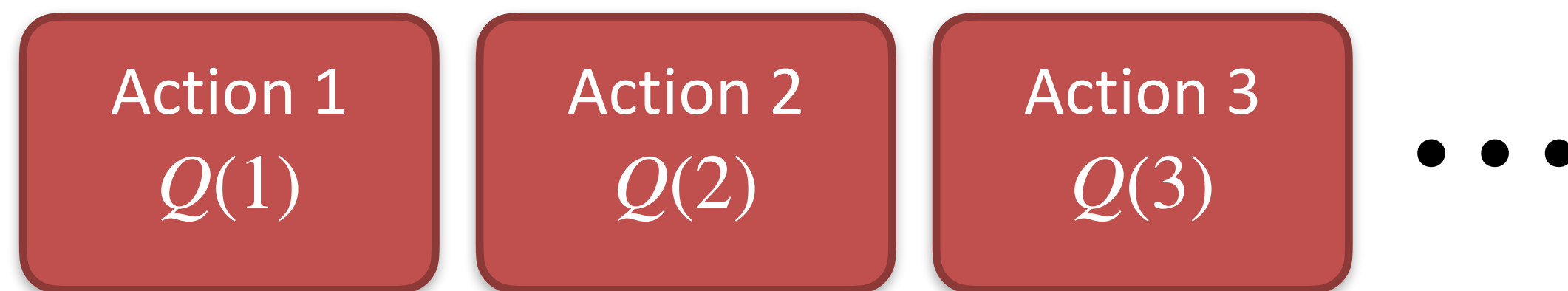
- UCB performs better than simple ϵ -greedy in case of the k -armed bandit problem
- However, it is difficult to apply to more general reinforcement learning problems



Learning to Choose Actions Directly

Approach so far

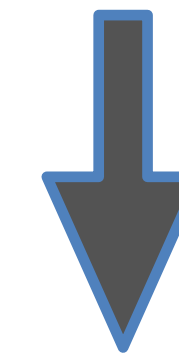
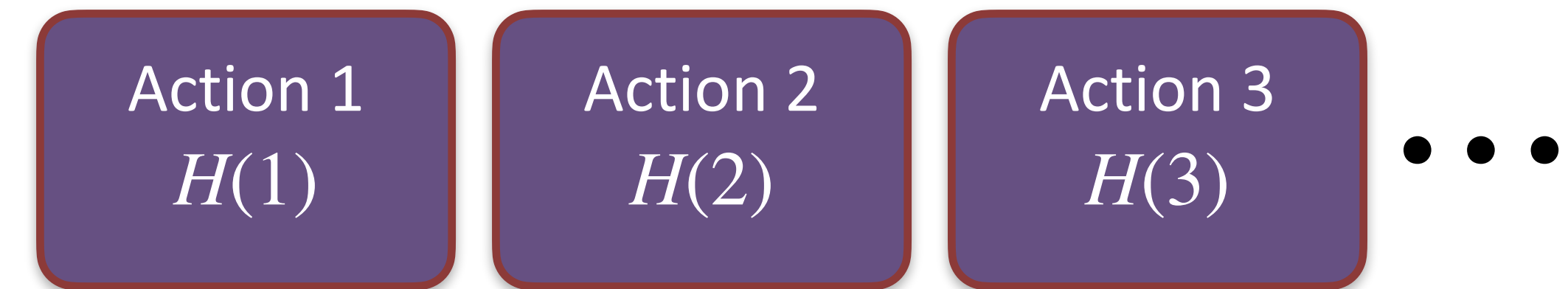
learn action values
(expected reward) for all actions



Choose action with highest estimated
value (greedy or ϵ -greedy)

Alternative approach

learn preference for all actions



Convert these preference scores to
probabilities

$$\mathbb{P}[a_i] = \frac{e^{H(i)}}{\sum_{j=1}^k e^{H(j)}}$$

How to Update the Preference Scores

Current probabilities

Action 1	Action 2	Action 3
0.20	0.50	0.30

Example 1: Choose Action 1 and receive a better than average reward

Action 1 $H(1) \uparrow$	Action 2 $H(2) \downarrow$	Action 3 $H(3) \downarrow$
-----------------------------	-------------------------------	-------------------------------

Action 1	Action 2	Action 3
0.30	0.45	0.25

Example 2: Choose Action 2 and receive a low reward

Action 1 $H(1) \uparrow$	Action 2 $H(2) \downarrow$	Action 3 $H(3) \uparrow$
-----------------------------	-------------------------------	-----------------------------

Action 1	Action 2	Action 3
0.25	0.40	0.35

Gradient ascent method

- Preference scores $H_t(a)$ towards action a
 - Variables: these are what we keep updating
- Choose actions based on a probability
 - Convert the preference scores to probability distribution using softmax
 - Define the probability of choosing action a as $\pi_t(a) = \mathbb{P}[A_t = a] = \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}}$
- Objective function to be maximized: expected reward $\sum_{x=1}^k \pi(x) q_*(x)$
- But we don't know $q_*(x)$, instead we can draw samples for $x = a$
 - High reward for action $a \rightarrow$ estimate for $q_*(a)$ increases $\rightarrow$ should increase $\pi(a)$
 - Objective function increases $\rightarrow$ parameter $H_t(a)$ increases
 - **Stochastic gradient ascent**

Bandit algorithm as gradient ascent

Our update rule

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)}$$

where

$$\mathbb{E}[R_t] = \sum_x \pi_t(x) q_*(x)$$

Derivation of stochastic gradient ascent

- Claim: $\frac{\partial \pi_t(x)}{\partial H_t(a)} = \pi_t(x)(\mathbf{1}_{a=x} - \pi_t(a))$, where $\mathbf{1}_{a=x} = 1$ if $a = x$, 0 otherwise

- Proof:
$$\frac{\partial \pi_t(x)}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left[\frac{e^{H_t(x)}}{\sum_{y=1}^k e^{H_t(y)}} \right] = \frac{\frac{\partial e^{H_t(x)}}{\partial H_t(a)} \sum_{y=1}^k e^{H_t(y)} - e^{H_t(x)} \frac{\partial \sum_{y=1}^k e^{H_t(y)}}{\partial H_t(a)}}{\left(\sum_{y=1}^k e^{H_t(y)} \right)^2}$$

$$= \frac{\mathbf{1}_{a=x} e^{H_t(x)} \sum_{y=1}^k e^{H_t(y)} - e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{y=1}^k e^{H_t(y)} \right)^2} = \frac{\mathbf{1}_{a=x} e^{H_t(x)}}{\sum_{y=1}^k e^{H_t(y)}} - \frac{e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{y=1}^k e^{H_t(y)} \right)^2}$$

$$= \mathbf{1}_{a=x} \pi_t(x) - \pi_t(x) \pi_t(a) = \pi_t(x)(\mathbf{1}_{a=x} - \pi_t(a))$$

$$\frac{\partial}{\partial w} \left[\frac{f}{g} \right] = \frac{g \frac{\partial f}{\partial w} - f \frac{\partial g}{\partial w}}{g^2}$$

because: $\pi_t(a) = P[A_t = a] = \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}}$

Derivation of stochastic gradient ascent

- Now we investigate the gradient $\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)}$ in our gradient ascent method
- We have
$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left[\sum_x \pi_t(x) q_*(x) \right] = \sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)}$$

$$= \sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} - \sum_x \frac{\partial \pi_t(x)}{\partial H_t(a)},$$
 because the sum $\sum_x \pi_t(x) = 1$, so its gradient is zero
- So, we can write the expression as $\sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} - B_t \sum_x \frac{\partial \pi_t(x)}{\partial H_t(a)}$ for any B_t independent of x
- Call B_t the baseline reward
- Hence, the gradient can be written as
$$\sum_x (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)}$$

Derivation of stochastic gradient ascent

- Next we multiply each term of the sum by $\frac{\pi_t(x)}{\pi_t(x)}$ and obtain the gradient as

$$\sum_x \pi_t(x)(q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} / \pi_t(x)$$

- This is in the form of an expectation, sum over all values of x , each term multiplied with $\pi_t(x)$
- So, it is equivalent to $\mathbb{E} \left[(q_*(A_t) - B_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right]$
- We don't know $q_*(A_t)$, but we have, $\mathbb{E}[R_t | A_t] = q_*(A_t)$
- Hence we use R_t as a replacement for $q_*(A_t)$ at timestep t
- As baseline B_t , one possible option is to take $\bar{R}_t$, average of all rewards so far

Derivation of stochastic gradient ascent

- Then we can write $\mathbb{E} \left[\left(q_*(A_t) - B_t \right) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] = \mathbb{E} \left[(R_t - \bar{R}_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right]$

- Since we know $\frac{\partial \pi_t(x)}{\partial H_t(a)} = \pi_t(x)(\mathbf{1}_{a=x} - \pi_t(a))$, we can write the gradient as

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E} \left[(R_t - \bar{R}_t) \pi_t(A_t) \left(\mathbf{1}_{a=A_t} - \pi_t(a) \right) / \pi_t(A_t) \right] = \mathbb{E} \left[(R_t - \bar{R}_t) \left(\mathbf{1}_{a=A_t} - \pi_t(a) \right) \right]$$

- Our stochastic gradient step: use *sample* of the expectation in place of the gradient

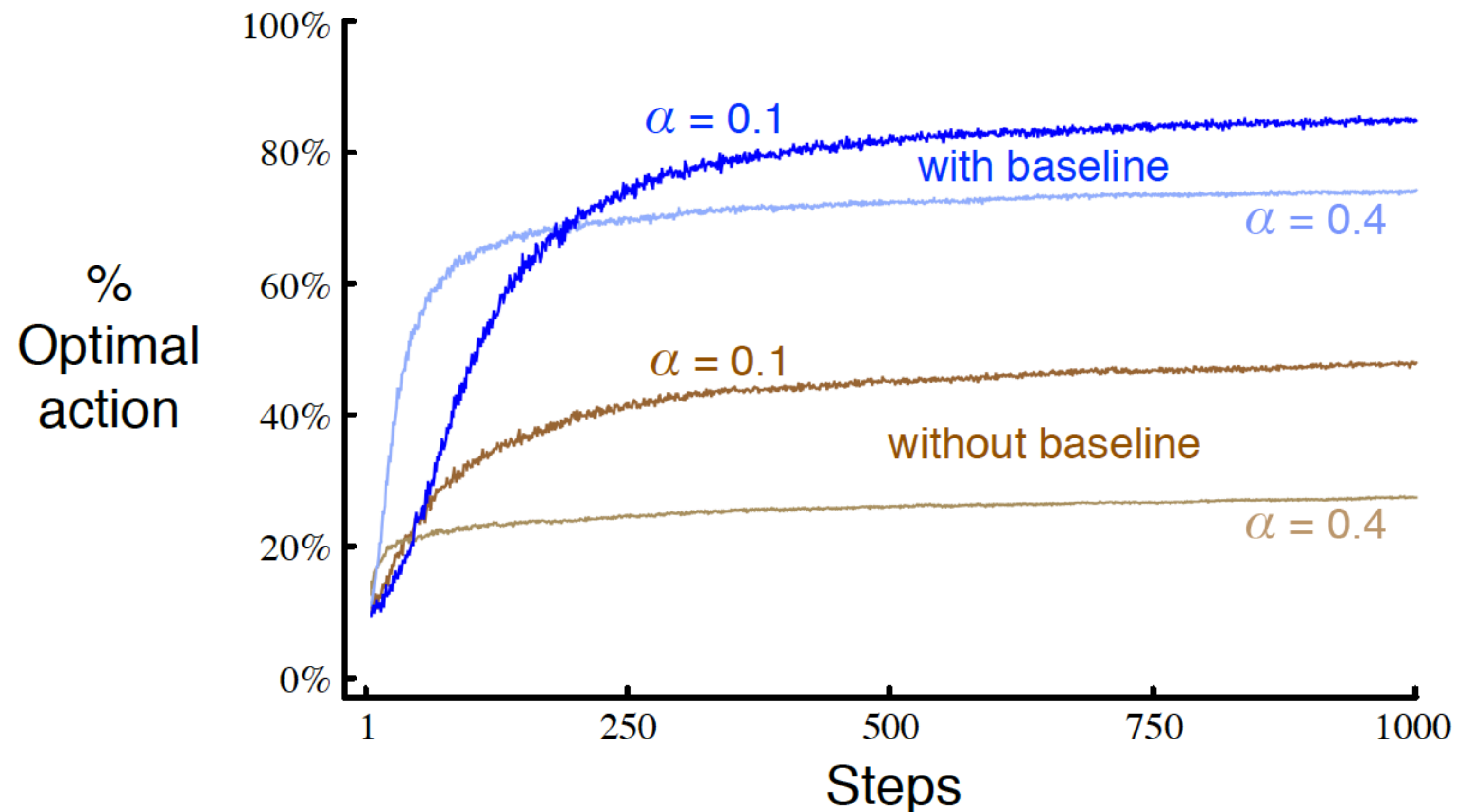
$$H_{t+1}(a) := H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} \simeq H_t(a) + \alpha (R_t - \bar{R}_t) (\mathbf{1}_{a=A_t} - \pi_t(a))$$

Gradient ascent : intuition

- Gradient ascent: $H_{t+1}(a) := H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbf{1}_{a=A_t} - \pi_t(a))$
- At each step t , after taking action $a = A_t$ and getting reward R_t , update:
$$H_{t+1}(A_t) := H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), \text{ and}$$
$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) \text{ for all } a \neq A_t$$
- If reward $R_t > \text{baseline } \bar{R}_t$, $H_t(A_t)$ is increased, otherwise decreased
- For all other actions, the opposite happens
- Choice of baseline B_t
 - Irrespective of the choice, as long as it is independent of the action (x), the algorithm would be a stochastic gradient ascent
 - However, the choice affects the variance and affects how the algorithm would converge

Gradient ascent : performance

- Performance on the 10-armed testbed (refer to the book)
- With baseline ($B_t = \bar{R}_t$) and without baseline ($B_t = 0$)



References

- Sutton, R.S. and Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press. <http://incompleteideas.net/book/the-book.html> (primary reference)
- Ashwin Rao. Multi-armed bandits: exploration vs exploitation. http://web.stanford.edu/class/cme241/lecture_slides/MultiArmedBandits.pdf